Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Web Security

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Threats to distributed systems
- Attack vectors
- Types of threat
- Identifying, Rating and Documenting threats
- Threat identification examples

UNIVERSITY of HULL

Web Security

- Principles of security in Distributed Systems and Applications:

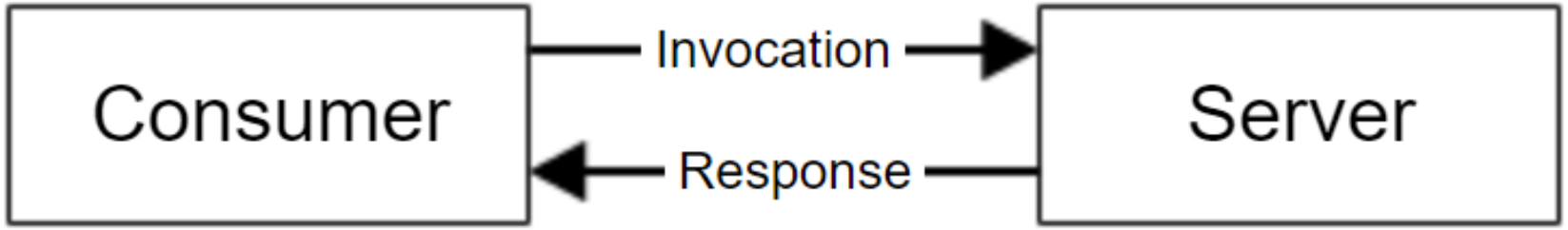
- Understanding security risks
- Documenting and modelling security threats
- Prevention of (minimising) security risks
  - Cryptography
  - Authorisation
  - Confidentiality
- Maximising chances of recovery after security event or compromise

UNIVERSITY of HULL

Security Threats to Distributed Systems

- The basic motivation of a Distributed System is to allow resource sharing between components

- As we've seen, components are typically thought of as Consumer and Server

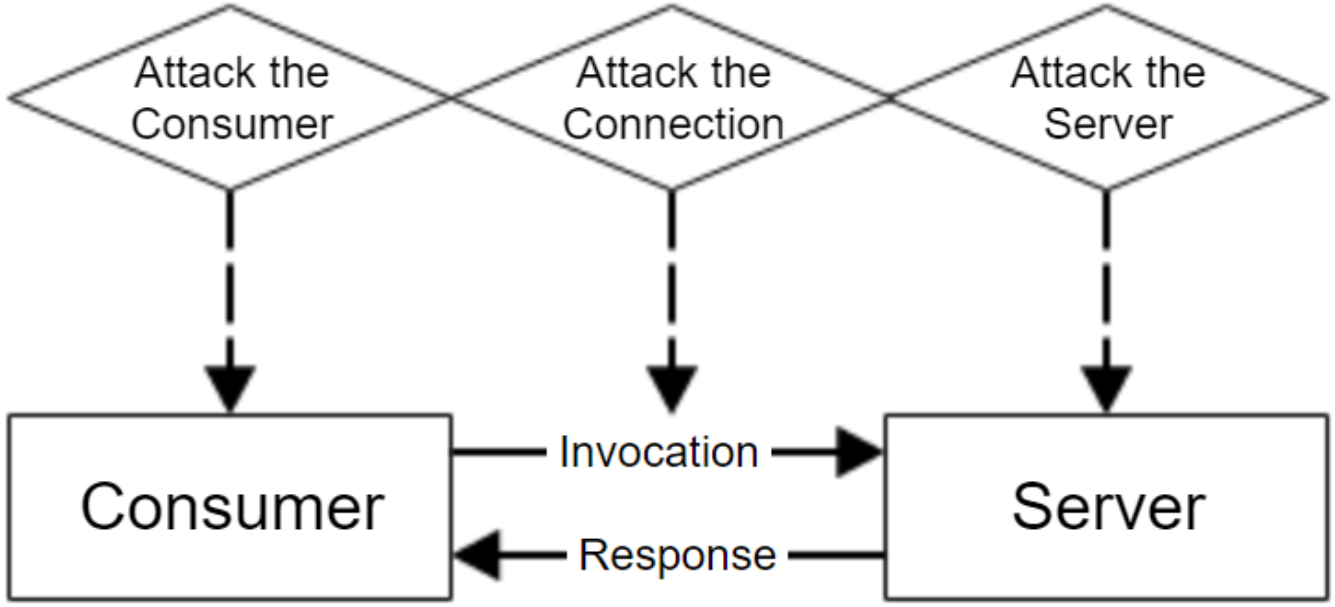


- So what part of this model do we need to protect?

UNIVERSITY of HULL

Distributed System Attack Vectors

- An attack could be made against any part of the system
- A system is only as strong as its weakest part
  - If the consumer is compromised, the server (who trusts the consumer) may become compromised too (and vice versa)
  - If the connection is compromised, the server and consumer may both become compromised or the data they are sending may become compromised.



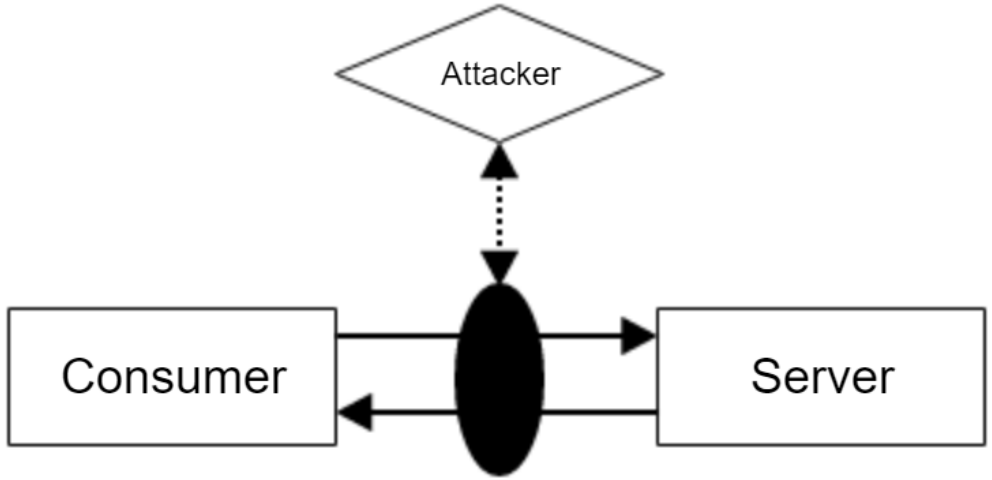
UNIVERSITY of HULL

Protecting the Connection

- Attacks on the connection often come in the form of a Man in the Middle (MITM) attack.

- Between the Consumer and Server sits a whole host of connections, routers and devices which we usually ignore unless something goes wrong. Internet traffic may go through hundreds of devices and use different routes, so it's often difficult to determine if there is an unscrupulous middleman.

- A MITM attack involves control or manipulation of a device which sits in between the two endpoints. This device could be planted there or exist there for (usually) good reasons (e.g. a router).



- Technologies like WiFi mean we can never be sure who is listening in or controlling the connection.

UNIVERSITY of HULL

Protecting the Server

- What the client sees as the server is usually more than one process on more than one machine

- Service(s) – may expose a number of services using different mechanisms
- Remote Service(s) – if you rely on another remote machine/service, it may be compromised
- Data Storage – may be on other machine(s)
- Operating System(s)
- Other software that connects to internet

- Weaknesses in any area of the Server might be exploited to gain access to data, manipulate the service or simply prevent access.

UNIVERSITY of HULL

Protecting the Client

- If you own both the client and the server, you can minimise risk by ensuring both have minimal risk.
- Usually, however, client machines are not owned or operated by you
- An often overlooked issue is that you cannot often guarantee that the client is not compromised
- Even if you have your own application running on the client side your system may still be attacked at the OS level or through other software
- Therefore you should always presume that the client has lowest trust and only allow clients to access and manipulate their own data

UNIVERSITY of HULL

Protecting the System – STRIDE Threats

- **S**poofing
  - False identity to gain access
- **T**ampering
  - Unauthorized modification of data
- **R**epudiation
  - Ability to deny having performed an action or a transaction
- **I**nformation disclosure
  - Exposure of private information, eavesdropping or traffic analysis
- **D**enial of service
  - Overloading of the server to deny the service to legitimate users
- **E**levation of privilege
  - User with limited access gains privileged access rights

UNIVERSITY of HULL

Server Trust

- Don’t allow your server to do more than it needs by itself
  - Server cannot perform ‘protected’ actions unless they are authorised by an authenticated user
  - Server cannot perform actions that it should never need to perform
- Track what a server is doing
  - Humans, AI and other monitoring software can spot unusual activity on the server/connection
- Lock down I/O access
  - Only accept incoming and outgoing connections from predefined allowed places
  - Only allow incoming and outgoing connections using predefined allowed protocols
  - Even a webserver should usually be behind an initial firewall
- Separate data and access
  - Keep data storage away from the internet
  - Ensure the server itself cannot decipher data

UNIVERSITY of HULL

Trusting the Client

- Use additional Authorisation and Authentication steps
  - 2FA / human intervention
  - More in the next lecture
- Don’t allow users to do things they shouldn’t need to do
  - Supply different software/access points to admins/users/guests
- Validate the client
  - Check the user is using the client
  - This is difficult (reverse engineering) but you can regularly update your validation process to minimise ‘fake’ clients
- Observe the client
  - Unusual behaviours could result in a lockout (AI, monitoring software, human)
- Validate the user’s machine
  - If you are running your client software on the machine, you can do your own audit
  - Check OS version, security patches, other software running, plugged in devices
  - Refuse to use the client if the machine fails

UNIVERSITY of HULL

Protecting the System – Physical and Social Threats

- Threats aren't only software/hardware based

- You will also need to be aware of physical threats
  - These involve physical access to the system or associated systems
  - Could be someone who is trusted acting maliciously
  - Could be someone not-trusted who has physical access to a machine (USB/Firewire ports, disk access, motherboard access, keyloggers)
  - Perhaps someone who leaves a 'trusted' device on a train

- and social threats
  - Phishing and social engineering
  - "I'll hurt/blackmail you if you don't give me your password"

UNIVERSITY of HULL

Architectural Considerations

In a multi-layered architecture it can be easier to assess **where** threats may occur

It is usually practical to address security concerns in (and between) each layer

- **Data flow**
  - What data flows between the components?
- **Trust boundaries**
  - Who / what do the components trust?
- **Firewall**
  - Data access trusts business logic
  - Database server trusts calls from web application
- **Entry points**
  - Open ports
  - Web applications
  - Management interfaces
- **Privileged code**
  - Encapsulation of secure resources and operations

UNIVERSITY of HULL

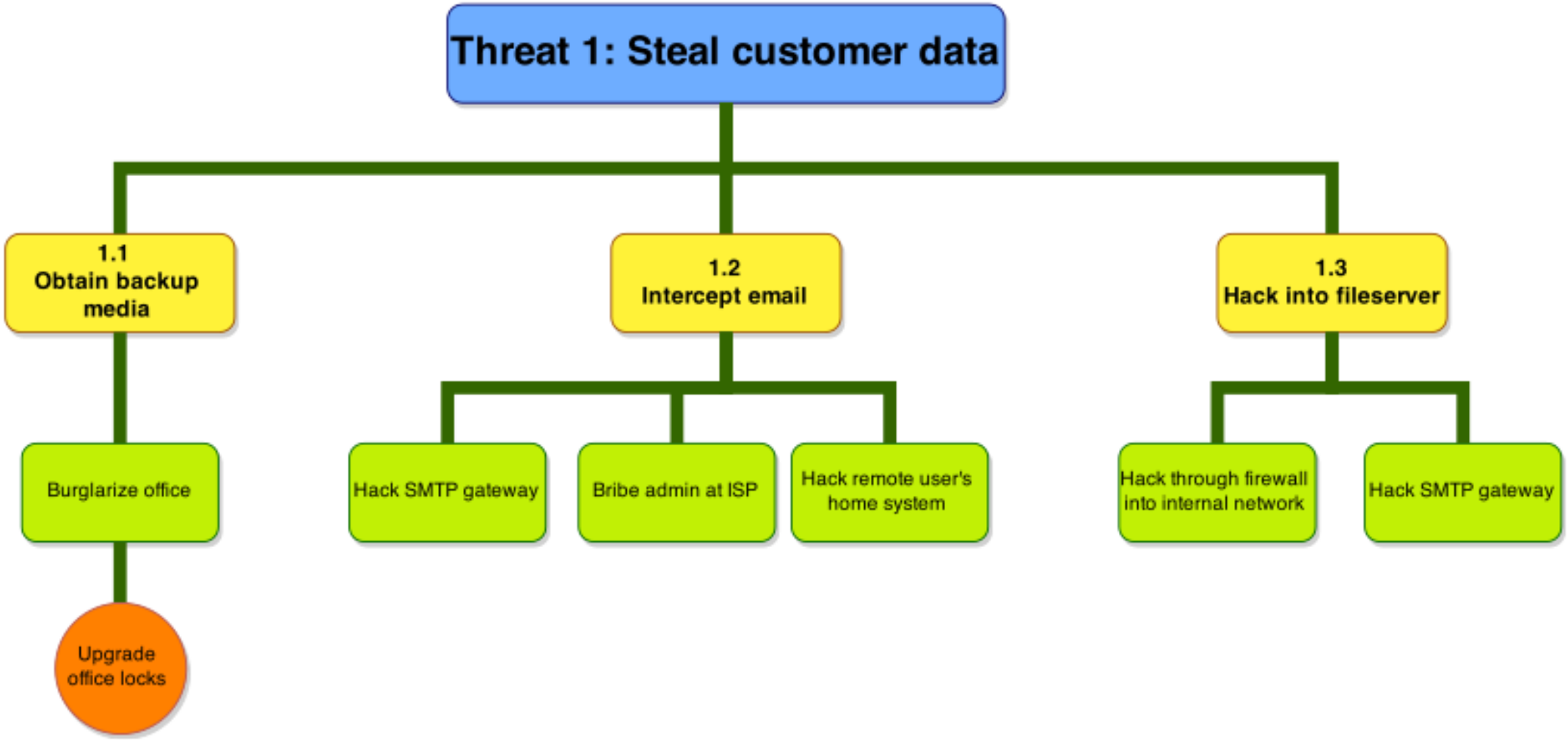
Documenting Threats: Threat Tables

Threat No.	1.1
Threat Description	Attacker obtains backup media
Threat Target Risk	Customer data
Attack Techniques	Burglarizing office
Countermeasures	Upgrade office locks

Threat No.	1.2
Threat Description	Attacker intercepts email
Threat Target Risk	Customer data
Attack Techniques	Hacking SMTP gateway, bribing admin at ISP, hacking remote user's home system
Countermeasures	??

Documenting Threats: Threat Trees

- An example (basic and incomplete) threat tree: Stealing Customer Data



Assessing Risk: DREAD Model

- Microsoft: DREAD model
  - Risk evaluation: (1) low, (2) medium, (3) high per each category
  - Sum of all categories: (12) and above is high risk, (8) and above is medium risk

- **D**amage potential
  - Damage severity if vulnerability is exploited
- **R**eproducibility
  - Difficulty of attack reproduction
- **E**xploitability
  - Difficulty of launching an attack
- **A**ffected users
  - Percentage of affected users
- **D**iscoverability
  - Difficulty of finding the vulnerability

UNIVERSITY of HULL

Assessing Risk: DREAD Model

Damage potential	Low (1): trivial leak Medium (2): sensitive information leak High (3): subvert the security, run code as administrator
Reproducibility	Low (1): difficult, need the knowledge of security hole Medium (2): timing window, particular race situation High (3): attack works every time
Exploitability	Low (1): extremely skilled person Medium (2): skilled programmer High (3): novice programmer
Affected Users	Low (1): few users, obscure feature Medium (2): some users, non-default configuration High (3): all users, default configuration
Discoverability	Low (1): obscure bug, unlikely to find Medium (2): infrequently used part of product, not obvious to exploit High (3): information about the attack published, commonly used functionality

Examining Unexpected Use-Cases

As a developer you create and follow:

Requirements / specification / designs

Therefore it is difficult to think outside the box…

"How might someone interact with my system in an unexpected way that may cause a security/privacy or other threat?"

This encompasses:

- The device(s) used and the software running on those devices
- The location and protections around the physical devices
- The network and messages sent to/from devices
- Data storage and backup

Usually systems/network engineers

- **The software you create and the behaviours/data you expose to your users**
- **The ways in which you permit authorisation and authentication**

UNIVERSITY of HULL

Real-World Example 1 – Ignore Payment

Real issue (currently not fixed!) but the specific details changed…

An organisation has a website and offers a driveway clearing service amongst other things

The organisation pays a software company for all of their website and specialist in-house software

- The software is generic so that the company can sell it to lots of organisations

The organisation wants to allow people to book a driveway clear from the website

- The software includes a form-filling component, so the software company extends this to allow payment. The process is this:

1. Form Page 1. Take customer details. On NEXT, send details to server and look up customer
2. Navigate to Form Page 2. If customer is a new customer, insert "new customer deal" price of £15.00 into form, otherwise insert price of £20.00. Search available dates/times and return to form. Take booking details (date/time). On NEXT, send all booking details to server to temporarily book slot.
3. Navigate to Form Page 3. Send price to payments processor and return the payments processor page.
4. Navigate to Payments Processor. When complete, confirmation of payment permanently books slot.
5. Navigate to Success page.

Any issues you can think of?

UNIVERSITY of HULL

Real-World Example 1 – Ignore Payment

There are lots of possible issues.

The one this organisation/software company didn't pick up was:

Because this was software originally designed for form-filling, all of the form entries are maintained on the client side until they are sent to the server. The server is stateless and wants to store nothing in memory about the client.

The only information stored in the database is: The client details, the booking data.

What's missing?

Real-World Example 1 – Ignore Payment

What's missing?

The price is managed IN THE FORM.

Yes, the server returns a price but this is then sent to the client, which sends it BACK to the server for it to direct the user to the payments processor. The client is responsible for remembering the price!

Yes, the price isn't editable on the UI but it is in the JavaScript/HTML!

So a malicious user changes the price to £0 and returns this instead.

The payment processor is then asked to process a payment of £0 (which it does by automatically returning success without asking for payment) and the server permanently books the slot.

This is a real exploit I identified and it isn't fixed!

UNIVERSITY of HULL

Real-World Example 2 – Who are you?

Real issue – identified at design time

I operate a timesheet payment portal for our demonstrators

Some of the functions on the website:

- Register with username and password
- Add and update personal information (e.g. name, contract number, email address)
- Add an entry to a timesheet for a single piece of work
- Check if an entry has been signed-off by staff
- Submit a timesheet that contains all of the entries you are claiming for
- Check if the timesheet has been processed for payment
- Timesheet is generated for the manager, inserting the user's name, contract number, email address and all of the entries that they have added to the timesheet. This automatically calculates the number of hours of work completed.

UNIVERSITY of HULL

Any possible issues you can identify?

Real-World Example 2 – Who are you?

Again, lots of possible issues!
One for focus: How does a member of staff know who is submitting for sign-off?

Staff know their demonstrators by name.

Staff are provided with the demonstrators name, the module information and the hours claimed so they can verify that the work was actually done.

Imagine that you are a demonstrator but you want to be paid for some extra time you didn't do…

1. Register and add your name as the name of another demonstrator who you know has lots of hours.
2. Add an entry to your timesheet immediately after that demonstrator finishes the work.
3. The staff member will see the entry and sign it off – the name is correct, and the hours were worked
4. Once you have done this, change your name on the site to your name and submit the timesheet. You now have an approved entry for hours you didn't do and your timesheet has YOUR name and contract number on it.

UNIVERSITY of HULL

Real-World Example 2 – Who are you?

Simple solution

Ensure that if someone changes their name, any currently unclaimed but approved entries are voided.

They need to be re-sent to the staff member for verification with the new name attached.

- Possibility of it being slightly annoying if you change your name (very unusual)
- Ensures that timesheets have the name of the person who actually claimed for the hours!

Name changes were added as a function for anyone who changed their name or made a mistake (e.g. the name needs to be the same as on their contract). Name changes seem to have nothing to do with whether someone has or has not done the work.

The only way this was spotted was design-time analysis!

UNIVERSITY of HULL

A Good Distributed System Developer…

- Recognises that threats/hacks/exploits *may* be inevitable
- Identifies as many possible threats and threat vectors as possible
- Designs the system to be easily maintained and quickly updatable/patchable
  - Layered architectures
  - Loose coupling
  - Abstraction
- Minimises risk by targeting all of the identified threats and putting protections in place
- Uses logging and system self-awareness to identify possible attacks
- Minimises risk by limiting access to only parts of the system

- Usually doesn't rewrite or develop new security-based functionality (unless the developer is an expert in that field)

UNIVERSITY of HULL

Testing

- Automated testing is key to providing assurances that your system/components can stand up to **identified** threats before you move to a production environment

- You've done all you can (/know how to) to minimise the threat
- But what about all the stuff you didn't think about or realise?

Successful attacks can be very expensive and damaging to your reputation:

- Leaks of personal data (data protection nightmare)
- The loss of functionality could bring service(s) to a halt (yours and clients')
- Malicious attacks/changes can be dangerous to users/clients
- Illicit activity can be hard to trace and could be up to you to compensate (e.g. bank)

UNIVERSITY of HULL

Penetration Testing

Human-led testing where damage potential is high is important:

- Specific focus on trying to
  - Access data without correct permissions
  - Destroy data without correct permissions
  - Prevent legitimate access to data
  - Take control over or access functionality of applications without correct permissions
  - Focus on every component and weak point in the system, not just your application

- In-house – do you know what you are looking for? Too close to it? No hackers employed?

  - Private penetration testers – 'White Hat Hackers' – Ethical Hackers
  - Bug bounties – make it more profitable to share than to steal/sell

UNIVERSITY of HULL

Further Reading

- ASP.NET CORE Security:
  - https://docs.microsoft.com/en-us/aspnet/core/security/
- William Stallings, Cryptography & Network Security
  - http://williamstallings.com/Cryptography/
- Ross Anderson, Security Engineering
  - http://www.cl.cam.ac.uk/~rja14/book.html

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Threats to distributed systems
- Attack vectors
- Types of threat
- Identifying, Rating and Documenting threats
- Threat identification examples

In the next session we will look at Policy, Authorisation, Authentication and Impersonation

UNIVERSITY of HULL